Sprawozdanie z projektu: Model programowego symulatora mikroprocesora

Kazimierz Szyszkowski Ewa Dziś

6 kwietnia 2026

1 Sformułowanie zadania i założenia szczegółowe

Celem projektu było napisanie aplikacji w standardzie PC, symulującej działanie prostego 16-bitowego mikroprocesora. Aplikacja została wyposażona w zaawansowany graficzny interfejs użytkownika (GUI) z wizualizacją LED, wzorowany na dostarczonym przykładzie, oraz realizuje wszystkie wymagane funkcje architektury, listy rozkazów i trybów adresowania.

1.1 Architektura procesora

Zgodnie z wymaganiami przyjęto następujący model:

- **Rejestry ogólnego przeznaczenia:** Cztery 16-bitowe rejestry o nazwach `A`, `B`, `C`, `D`.
- **Podział rejestrów:** Każdy z głównych rejestrów (`A`, `B`, `C`, `D`) można traktować jako parę 8-bitowych rejestrów (starsza część `H` i młodsza `L`), co daje dostęp do rejestrów `AH`, `AL`, `BH`, `BL`, `CH`, `CL`, `DH`, `DL`.

1.2 Lista rozkazów i tryby adresowania

- **Dostępne rozkazy:**
  - `MOV dest, src` – przesłanie danych z `src` do `dest`.
  - `ADD dest, src` – dodawanie (`dest = dest + src`).
  - `SUB dest, src` – odejmowanie (`dest = dest - src`).
- **Tryby adresowania:**
  - Rejestrowy (np. `MOV AX, BX`) – operandami są rejestry (użyto nazw ze zdjęcia lub nazw 8-bitowych).
  - Natychmiastowy (np. `ADD AL, 0x15`) – jednym z operandów jest bezpośrednia wartość liczbowa (obsługiwany format dziesiętny lub szesnastkowy z przedrostkiem `0x`).

1.3 Składnia i interfejs

Przyjęto składnię asemblera Intel (destynacja, źródło) rozdzieloną przecinkiem. Separatorami są spacje oraz przecinek. Program wyposażono w graficzny interfejs (`tkinter`), podzielony na:

- **Panel Lewy (Wizualny):** Wizualizacja rejestrów z diodami LED (16-bit, od bocznej 15 do 0), polami binarnymi, etykietami H/L i przyciskami "WPISZ". Dedykowana sekcja wizualizacji dla argumentu natychmiastowego. Pole wejściowe instrukcji.
- **Panel Prawy (Tekstowy/Sterowanie):** Lista kodów (edytor programu) z paskiem przewijania i numeracją linii. Przyciski sterujące: wykonanie ciągłe, krokowe (z podświetleniem linii), reset, zapis i odczyt z pliku tekstowego (`.txt`).

2 Opis przyjętych rozwiązań programowych

Aplikacja została napisana w języku Python zgodnie z paradygmatem obiektowym. Logika mikroprocesora została odseparowana od warstwy wizualnej, co ułatwia ewentualną rozbudowę.

2.1 Struktura danych dla rejestrów i współdzielenie pamięci

Aby zrealizować wymóg współdzielenia pamięci dla rejestrów 16-bitowych i 8-bitowych (np. zmiana `AL` musi modyfikować dolne 8 bitów `AX`), zdecydowano się przechowywać stan procesora jako słownik posiadający tylko rejestry 16-bitowe (`A`, `B`, `C`, `D`). Dostęp do części 8-bitowych rozwiązano za pomocą operacji bitowych w metodach `get_val` i `set_val`.

Listing 1: Fragment metody `set_val` zarządzającej zapisem do rejestru (współdzielenie pamięci)

```python
def set_val(self, operand, value):
    value = value & 0xFFFF # maskowanie do 16 bit

    if operand in self.regs: # AX, BX, CX, DX
        self.regs[operand] = value
    elif len(operand) == 2 and operand[1] in 'HL': # AH, AL...
        base_reg = operand[0] # Pobierz litere A, B, C lub D
        current_val = self.regs[base_reg]
        if operand[1] == 'H':
            # Zeruj starszy bajt i wstaw nowa wartosc
            self.regs[base_reg] = (current_val & 0x00FF) | ((value & 0xFF) << 8)
        else:
            # Zeruj mlodszy bajt i wstaw nowa wartosc
            self.regs[base_reg] = (current_val & 0xFF00) | (value & 0xFF)
```

2.2 Parsing linii programu

Do analizy składniowej (parsingu) wprowadzanych instrukcji tekstowych użyto wyrażeń regularnych (moduł `re`). Pozwala to na elastyczne ignorowanie nadmiarowych spacji oraz

łatwe wyodrębnienie polecenia i operandów, bez konieczności tworzenia sztywnego interfejsu z menu rozwijanymi, co ograniczałoby funkcjonalność.

Listing 2: Parsing pojedynczej instrukcji za pomocą `regex`

```
# Rozbijanie instrukcji np: "MOV AX, 0x1A" na grupy
match = re.match(r'([A-Z]+)\s+([A-Z0-9]+)\s*,\s*([A-Z0-9x]+)',
    instruction, re.IGNORECASE)
if not match:
    raise ValueError("Blad skladni")
cmd, dest, src = match.groups()
cmd = cmd.upper()
src_val = self.get_val(src)
```

2.3 Graficzny interfejs z wizualizacją LED i podglądem argumentu natychmiastowego

Interfejs graficzny został zrealizowany przy użyciu biblioteki `tkinter`. Zastosowano pomocnicze metody (np. `create_led_row`) do automatyzacji budowy paneli z diodami dla każdego rejestru. Wartości binarne w polach tekstowych są aktualizowane przy każdym kroku lub zmianie rejestru. Dedykowana sekcja wizualizacji dla argumentu natychmiastowego jest aktualizowana dynamicznie, gdy użytkownik zaznaczy instrukcję na liście kodów lub gdy instrukcja jest wykonywana w trybie krokowym.

3 Dyskusja wyników

Opracowana aplikacja w pełni realizuje zadanie projektowe i jest funkcjonalnie poprawna. Graficzny interfejs użytkownika jest dopracowany, intuicyjny i znacznie uatrakcyjniony dzięki wizualizacji diod LED, wzorowanej na dostarczonym przykładzie, co czyni symulację bardziej czytelną.

Zalety aplikacji:

- Zaawansowany, czytelny interfejs graficzny z wizualizacją LED dla rejestrów i argumentu natychmiastowego, ułatwiający naukę i śledzenie stanu bitów.
- Prawidłowa obsługa trybów H/L z użyciem masek bitowych, gwarantująca integrację zmienianych części 8-bitowych z rejestrem 16-bitowym.
- Zastosowanie parsingu tekstowego, które zapewnia pełną elastyczność i realizację wszystkich wymagań, bez sztywnych ograniczeń interfejsu ze zdjęciem.
- Pełna obsługa wejścia/wyjścia na plikach (odczyt i zapis skryptów).
- Precyzyjne śledzenie trybu krokowego z podświetleniem linii.

Wady i ograniczenia:

- Model nie uwzględnia obsługi flag stanu (np. *Carry Flag, Zero Flag, Sign Flag*), przez co wynikom operacji arytmetycznych nie towarzyszy informacja o przepełnieniu znakowym/bez znakowym.
- Brak zaimplementowanych instrukcji skoków (warunkowych i bezwarunkowych) ogranicza pisanie zaawansowanych algorytmów (np. pętli).

- Przyciski "WPISZ"obok rejestrów są w pewnym sensie nielogiczne w kontekście edytora kodu po prawej, ale zostały dodane w celu zachowania zgodności wizualnej ze zdjęciem.

Podsumowując, napisany symulator stanowi doskonałą platformę dydaktyczną do zrozumienia podstawowych mechanizmów działania procesora, przepływu danych pomiędzy rejestrami oraz wizualizacji ich binarnej zawartości.